Innovation partner H2S HACK2SKILL

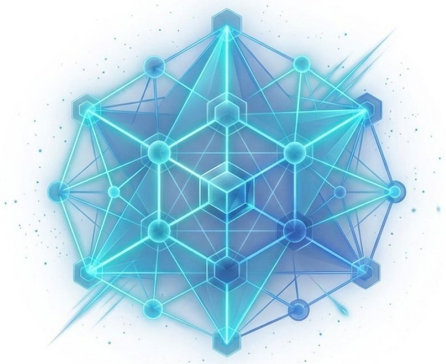
Media partner YOURSTORY

AI for Bharat Hackathon

Powered by aws

Project Title: Fragility Graph

Team Name : Phantom Troup



FRAGILITYGRAPH

Real-time Structural Debt Prediction using Graph Neural Networks

Predicting code breaks before they happen

The $1 Trillion Problem: "The Ripple Effect"

"A single line change in a utility function can cascade into 14 breaking microservices - and you only find out in production."

Modern software development is plagued by "The Ripple Effect" where code changes in one area silently break functionality in another due to hidden dependencies that standard tools cannot detect.

The Gap:

- Current tools are reactive - they tell you something is broken AFTER build failures or production bugs
- Static analysis (linters/compilers) cannot see implicit dependencies

The Cost:

- "Structural Debt" makes developers afraid to refactor
- This "fear-driven development" slows feature velocity by up to 40% in mature projects
- Teams spend excessive time debugging ripple effects instead of building features

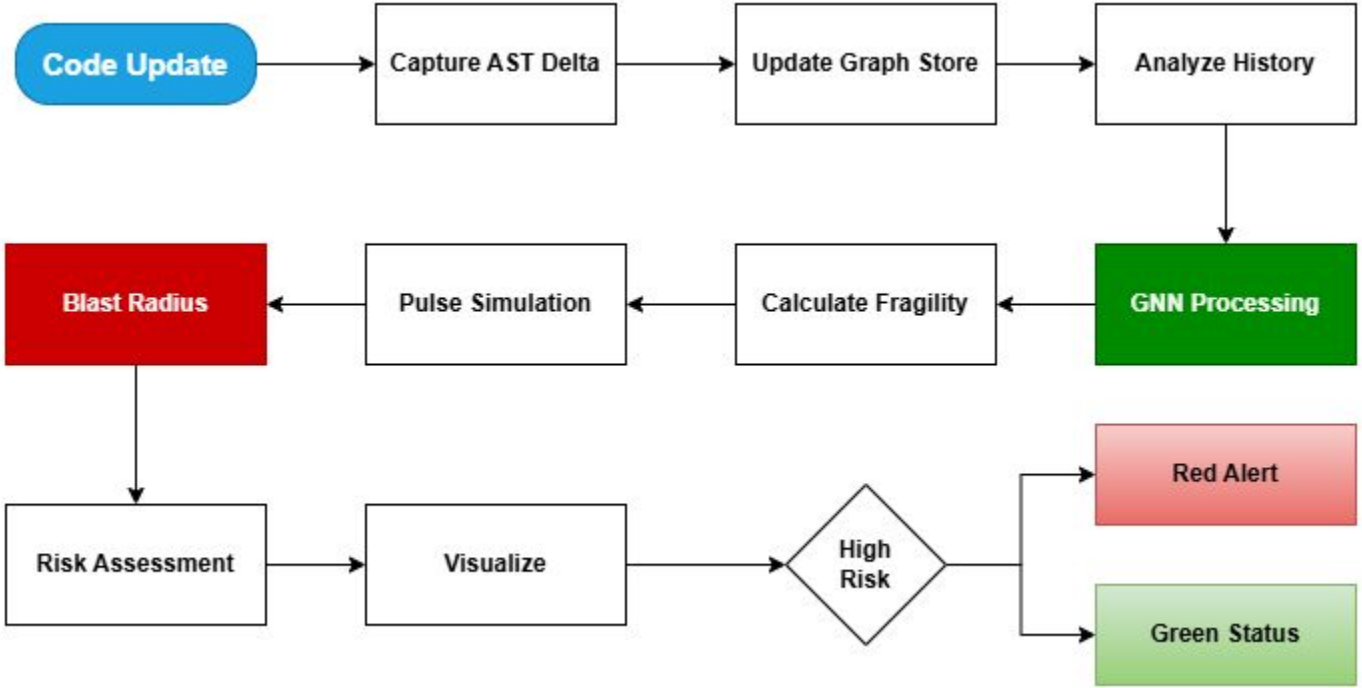
Our Solution: FragilityGraph

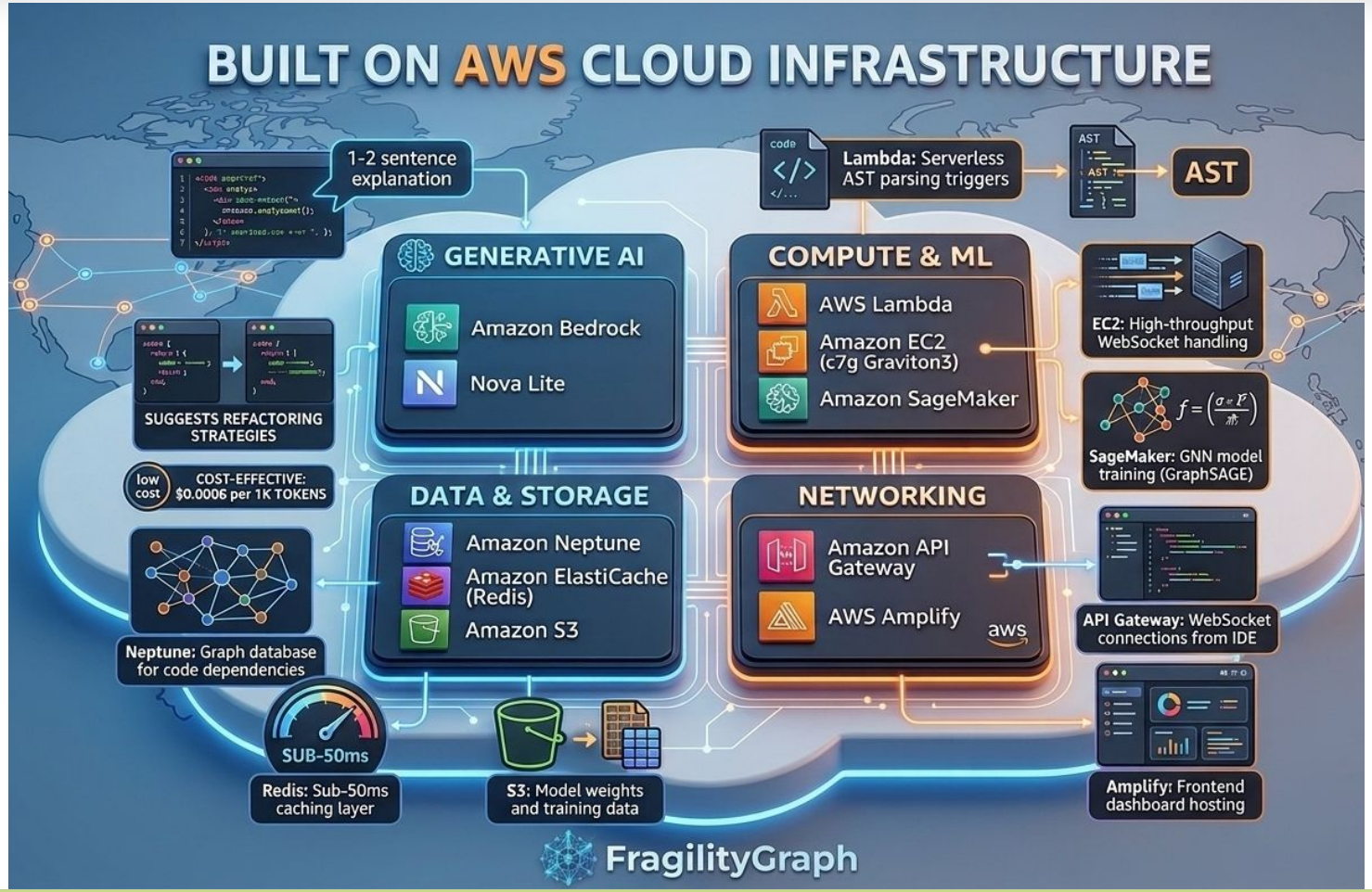
From Reactive to Proactive

Traditional Approach	**FragilityGraph Approach**
Static analysis	Graph-based structural analysis
Unit tests (post-build)	Real-time prediction as you type
Integration tests (post-deployment)	Machine learning fragility scoring
Discovers problems AFTER deployment	Predicts problems DURING development

Shift from 'What broke?' to 'What WILL break?

How It Works - Technical Workflow




BUILT ON AWS CLOUD INFRASTRUCTURE
1-2 sentence explanation
Lambda: Serverless AST parsing triggers
AST
GENERATIVE AI
Amazon Bedrock
Nova Lite
COMPUTE & ML
AWS Lambda
Amazon EC2 (c7g Graviton3)
Amazon SageMaker
EC2: High-throughput WebSocket handling
SUGGESTS REFACTORING STRATEGIES
low cost
COST-EFFECTIVE: $0.0006 per 1K TOKENS
SageMaker: GNN model training (GraphSAGE)
DATA & STORAGE
Amazon Neptune
Amazon ElastiCache (Redis)
Amazon S3
NETWORKING
Amazon API Gateway
AWS Amplify
aws
API Gateway: WebSocket connections from IDE
Neptune: Graph database for code dependencies
SUB-50ms
Redis: Sub-50ms caching layer
S3: Model weights and training data
Amplify: Frontend dashboard hosting
FragilityGraph

GenAI Model - Amazon Bedrock Nova Lite

Why Amazon Bedrock Nova Lite?

Key Features:

- **Fast**: <1 second inference time
- **Cost-effective**: 10x cheaper than larger LLMs
- **Context-aware**: Analyzes code structure + historical patterns
- **Production-ready**: Generates concise, actionable insights

Use Cases:

1. **Impact Explanations**: "This function authenticates 47 API endpoints and lacks retry logic, making any change likely to cause cascading login failures."
2. **Refactoring Suggestions**: "Consider wrapping in a circuit breaker pattern before refactoring."
3. **Onboarding Guidance**: "High-risk area - used by 12 critical services. Test thoroughly before changes."

Prompt Engineering:

- Strict 1-2 sentence constraint
- Technical but accessible language
- Specific about what will break and why
- Includes mitigation steps

Example Output Box:

Input: Function "*validate_session*", Fragility: 87/100
Output: "This function directly authenticates 47 API endpoints and lacks retry logic, making any change likely to cause cascading login failures. Consider wrapping in a circuit breaker pattern."

Architecture of the proposed solution:

LAYER 1: CLIENT LAYER
- IDE Extension (VS Code / JetBrains)
- User interface and interaction

LAYER 2: SENSOR LAYER
- Tree-sitter Parser (incremental AST extraction)
- AST Delta Extractor (tracks changes)

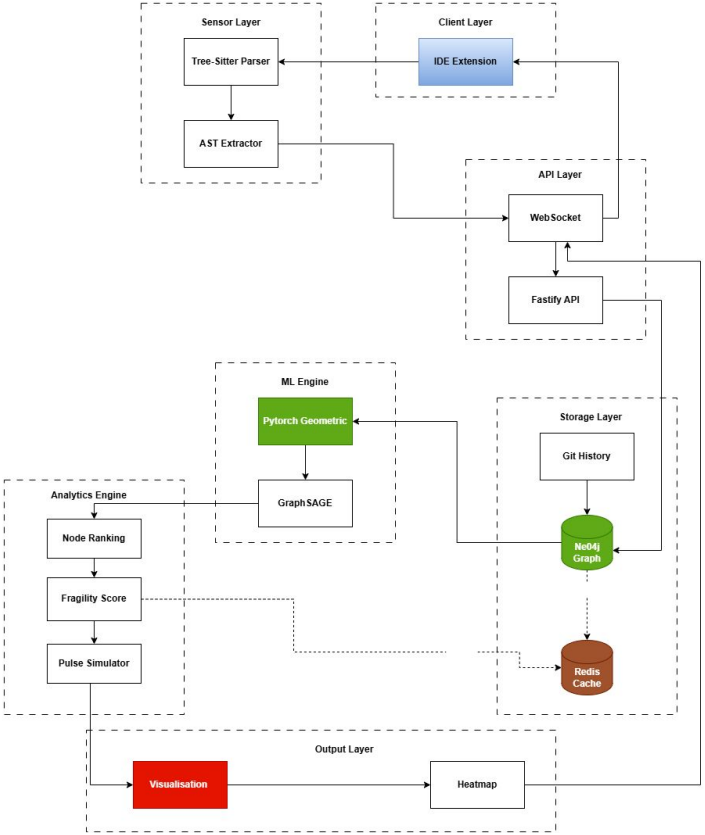
LAYER 3: API LAYER
- Fastify API Server (Node.js high-speed bridge)
- WebSocket Server (real-time bidirectional communication)

LAYER 4: STORAGE LAYER
- Neo4j Graph Database (nodes = classes, edges = dependencies)
- Redis Cache (fast lookups, model predictions)
- Git History Integration (temporal analysis)

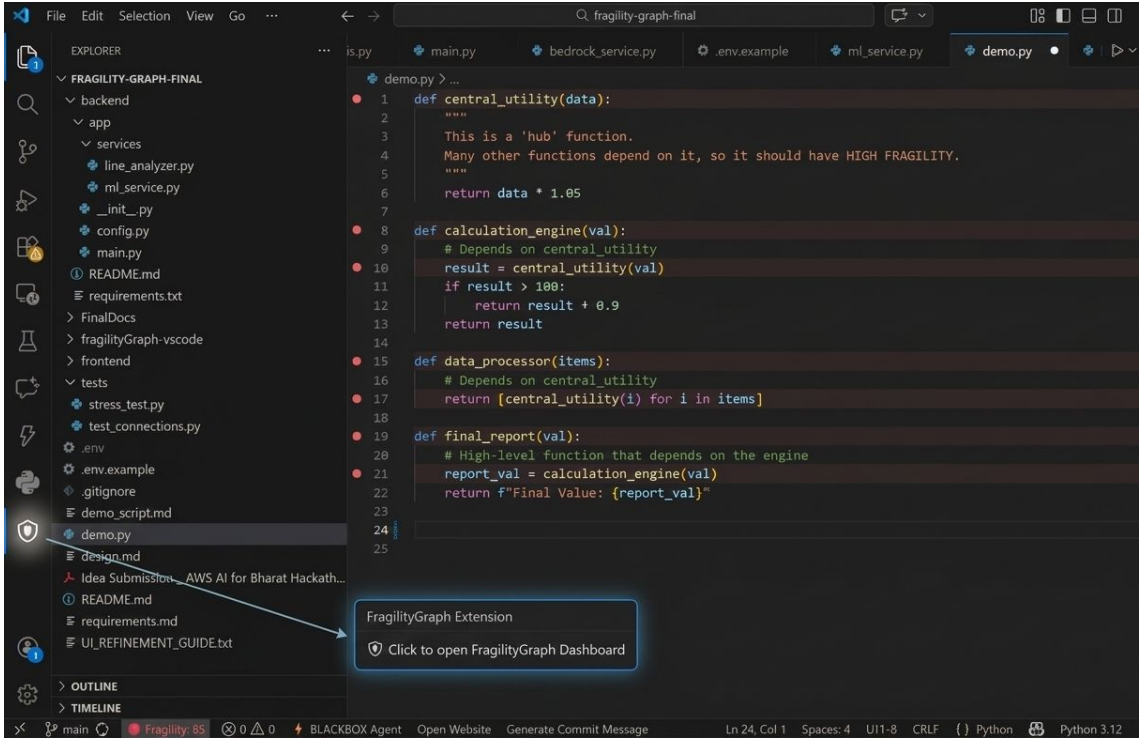
LAYER 5: ML ENGINE
- PyTorch Geometric (graph neural network framework)
- GraphSAGE Model (node embedding and learning)

LAYER 6: ANALYSIS ENGINE
- Node Importance Ranking (identifies critical components)
- Fragility Score Calculator (computes risk metrics)
- Pulse Simulator (propagation modeling)

LAYER 7: OUTPUT LAYER
- Visualization Generator (creates visual representations)
- Heatmap Renderer (shows risk distribution)

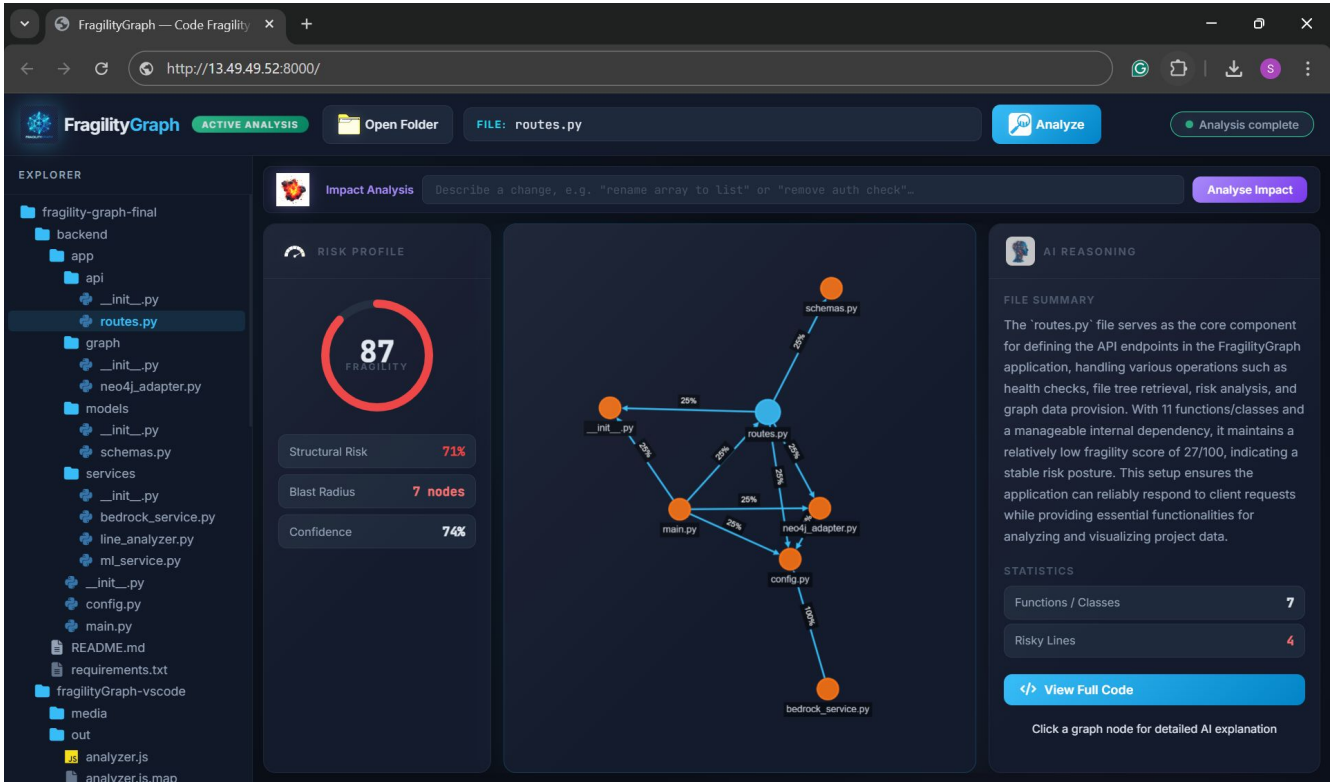


VS Code Extension



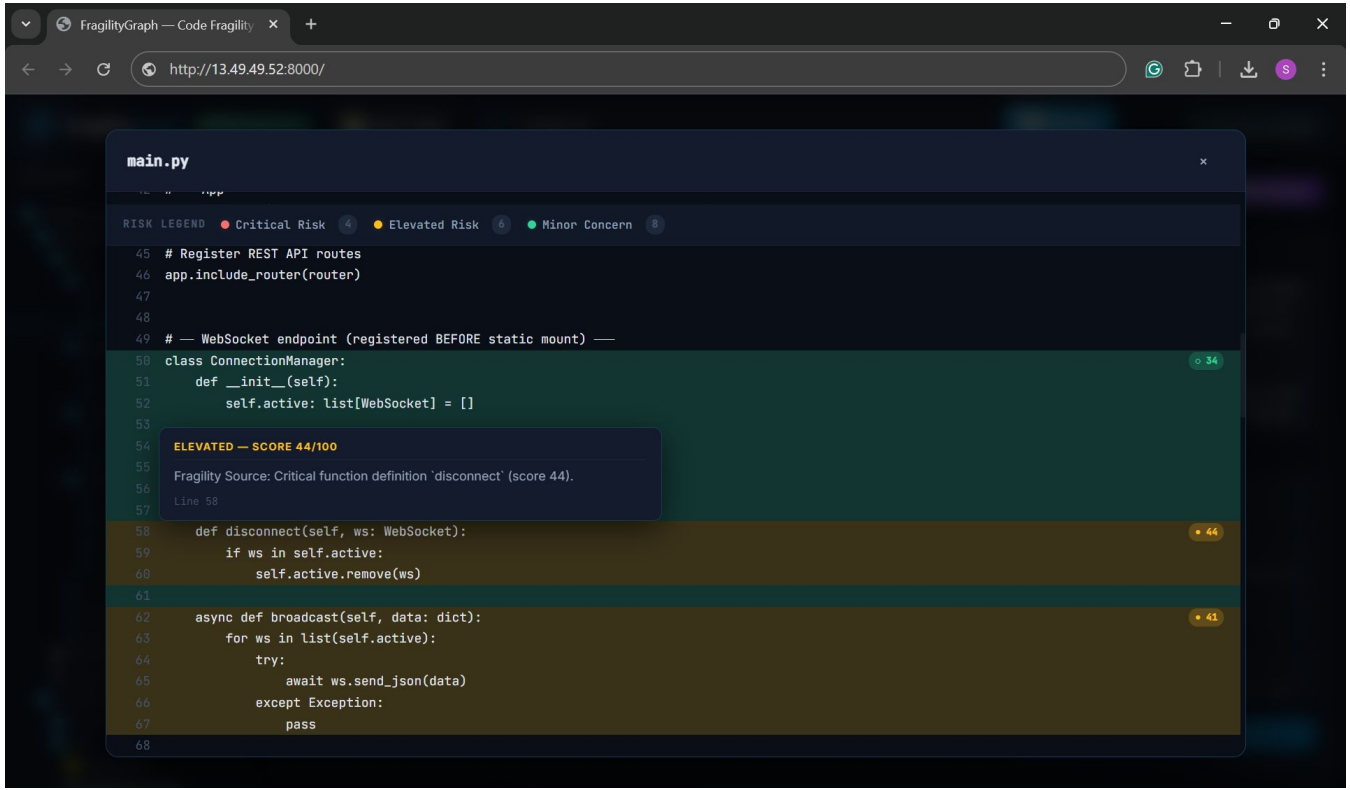
See exactly which lines are dangerous before the file is even saved

Web Dashboard



Real-time visualization of code fragility and dependency risk

Web Dashboard



Line-level fragility detection with AI-powered explanations

Key Features & Differentiators

What Makes FragilityGraph Different?

Real-time Impact Analysis ● Instant feedback as you type (<2 seconds) ● No waiting for builds or tests ● Predicts breaks across entire codebase	**GNN-Powered Predictions** ● Structural analysis, not just pattern matching ● Learns from historical failures ● 75%+ accuracy on test datasets
AI Explanations ● Amazon Bedrock Nova Lite generates insights ● Short, production-ready explanations ● Actionable mitigation suggestions	**Team Collaboration** ● Shared dashboards for visibility ● Onboarding heatmaps for new developers ● Prioritized refactoring roadmaps

Technical Metrics & Performance

Performance Metrics: • API Latency: <2 seconds end-to-end • WebSocket Latency: <100ms message delivery • Cache Hit Rate: 80%+ (ElastiCache) • Graph Query Time: <200ms (Neptune) • GNN Inference: <500ms per prediction	**Scalability:** • Codebase Size: Up to 1M lines of code • Concurrent Developers: 100+ users • Graph Size: 100K+ nodes, 500K+ edges • Daily Analyses: 10K+ code changes
Cost Efficiency: • Development: $10-20 (one-time training) • Production: $14K/year for 100 developers • Per Developer: $10-15/month • ROI: 5x (developer time saved vs. cost)	**Success Criteria:** ✅ 60% reduction in regression bugs ✅ 25% faster refactoring cycles ✅ 40% improvement in developer confidence

Implementation Cost Breakdown

Development Phase

Service	Configuration	Cost
AWS Lambda	100K Invocations	$0 (free tier)
DynamoDB	1GB data	$0 (free tier)
S3	5GB Storage	$0 (free tier)
SageMaker Training	ml.m5.large × 2 hours	$0.30 (one-time)
CloudWatch	Basic Logs	$0 (free tier)
Total Dev Cost		$10-20

Production Phase (100 developers)

Service	Configuration	Monthly Cost
AWS Lambda	1M Invocations	$0.20
DynamoDB	25GB data	$6.25
Nepture	db.r5.large	$350
ElastiCache	cache.r6g.large	$120
SageMaker Inference	Serverless (Lambda)	$0
Bedrock	500K tokens/day	$100
Total Monthly		~$600

Security & Compliance

Enterprise-Grade Security

Encryption: ● **At Rest**: All data encrypted in Neptune, DynamoDB, S3 (AWS KMS) ● **In Transit**: TLS 1.3 for all connections (HTTPS, WSS) ● **API Keys**: IAM-based authentication	**Data Privacy**: ● **Code Never Leaves Infrastructure**: On-premise deployment option ● **VPC Isolation**: All services in private VPC ● **No Third-party Sharing**: Code analysis stays internal
Compliance: ● **GDPR Ready**: Data retention policies configurable ● **SOC2 Compatible**: Audit logs via CloudWatch ● **Role-Based Access**: AWS IAM integration	**Access Control**: ● Developer: View own code analysis ● Tech Lead: Team-wide dashboards ● Admin: System configuration and user management

The Future of Coding is Fearless

From reactive debugging to proactive prediction.
From fear-driven development to confident refactoring.
From hidden failures to transparent risk

FragilityGraph: *Break Nothing*

Contact: seetharamkkv@gmail.com

Innovation partner H2S HACK2SKILL | Media partner YOURSTORY

AI for Bharat Hackathon

Powered by aws

Thank You